



But

Dans ce chapitre présente diverses façons d'acheminer et de récupérer de la donnée en allant de la plus commune avec HTTP à la plus spécifique dans le monde de l'IoT avec le protocole LoRaWAN en passant par le protocole MQTT. Nous concluons par une grille permettant de choisir le protocole et l'architecture IoT en fonctions de diverses contraintes : autonomie, débit, portée...

PLAN

- Récupérer de la donnée par http
 - Format JSON
 - Désérialiser un JSON
- Envoyer de la donnée par MQTT
 - Introduction rapide à MQTT
 - Utilisation dans l'environnement Arduino
- Récupérer de la donnée par WebSocket
 - Protocole WebSocket
 - MQTT sur WebSocket
- Envoyer de la donnée en LoRaWAN
 - Principes des réseaux LoRaWAN
 - Utilisation dans l'environnement Arduino



Récupérer de la donnée par http



Récupérer de la donnée par http

HTTP (Hypertext Transfer Protocol) est un protocole de communication client-serveur qui définit la manière dont les messages sont formatés et transmis dans le web. Les clients, en général des navigateurs, envoient des requêtes HTTP aux serveurs pour récupérer des ressources telles que des pages Web, des images, des fichiers ou des données. Les serveurs répondent à ces requêtes en renvoyant des réponses HTTP qui contiennent les informations demandées.

Récupérer de la donnée par http

Les requêtes HTTP sont composées de plusieurs éléments, notamment l'URL (Uniform Resource Locator) qui identifie la ressource demandée, les méthodes telles que GET (pour récupérer une ressource), POST (pour envoyer des données à un serveur), PUT (pour mettre à jour une ressource) ou DELETE (pour supprimer une ressource) ainsi que d'autres informations.

Récupérer de la donnée par http

Les réponses HTTP renvoyées par le serveur contiennent en plus des données d'autres éléments comme le code de réponse HTTP (200 pour une réponse réussie, 404 pour une ressource introuvable, etc.), des en-têtes qui contiennent des informations supplémentaires sur la réponse. Par exemple, le code suivant utilise la librairie Arduino HTTPClient . Il instancie un client et initialise une connexion HTTP. Le code de réponse HTTP est stocké dans la variable `httpCode`.

Récupérer de la donnée par http

```
1 #include <HTTPClient.h>
2
3 HTTPClient client;
4
5 client.begin("https://api.coindesk.com/v1/bpi/currentprice.json");
6 int httpCode = client.GET();
```

Format JSON

Dans l'IoT, les capteurs et les objets connectés produisent une grande quantité de données qui doivent être collectées, stockées et traitées. Le format JSON est très utilisé pour représenter ces données.

? Qu'est ce que le JSON ?

JSON (*JavaScript Object Notation*) est un format de données textuel d'échange de données. Un fichier JSON représente un tableau associatif similaire à un dictionnaire Python.

Format JSON

Voici un extrait d'un fichier JSON qui contient le dernier prix du Bitcoin qui vous sera également utile pour les exercices de fin de chapitre :

```
{
  "time": {
    "updated": "May 4, 2023 14:37:00 UTC"
  },
  "chartName": "Bitcoin",
  "bpi": {
    "GBP": {
      "code": "GBP",
      "symbol": "£pound;",
      "rate": "24,140.5197",
      "description": "British Pound Sterling",
      "rate_float": 24140.5197
    },
    "EUR": {
      "code": "EUR",
      "symbol": "€euro;",
      "rate": "28,143.3891",
      "description": "Euro",
      "rate_float": 28143.3891
    }
  }
}
```

Désérialiser un JSON

Pour parcourir et extraire des données d'un JSON, nous utiliserons la librairie ArduinoJson. C'est la librairie de référence : elle va nous permettre de sérialiser et surtout de désérialiser le contenu du JSON.

? Désérialiser un JSON ?

La désérialisation consiste en convertir une suite d'octets en une représentation mémoire. Dans notre cas, il s'agit de convertir un JSON en une structure de données C++ utilisable par notre programme afin d'en extraire les informations utiles.

Désérialiser un JSON

À titre d'illustration, nous allons utiliser la réponse du site

<https://api.coindesk.com/v1/bpi/currentprice.json> .

Ce fichier, mis à jour, toutes les minutes contient les derniers prix du Bitcoin.

Désérialiser un JSON

Stockage du JSON en mémoire

Dans un premier temps, nous stockons simplement le fichier JSON en mémoire dans un objet de type chaîne de caractères.

```
#include <HTTPClient.h>

HTTPClient client;

client.begin("https://api.coindesk.com/v1/bpi/currentprice.json");
int httpCode = client.GET();

String input = client.getString();
Serial.println(input);
```

Désérialiser un JSON

À ce stade, ce n'est pas la **représentation d'un JSON** avec sa structure hiérarchique mais simplement une chaîne de caractères à plat. Par exemple, le caractère accolade ouvrante n'a pas le sens de début de groupe qu'il a en JSON. Il est simplement vu comme un simple caractère. La librairie **ArduinoJSON** utilise une allocation mémoire de taille fixe et impose de préalablement **créer** un **JsonDocument** afin de réserver l'espace nécessaire en mémoire. Pour cela, le site <https://arduinojson.org/v6/assistant/> permet de calculer la place nécessaire. À noter que l'on peut prendre une taille inférieure mais cela tronquera les données.

Désérialiser un JSON

La ligne ci-dessous crée un document JSON nommé doc de 873 octets :

```
// On crée un document JSON doc (réservation mémoire)
StaticJsonDocument<873> doc;
```

Désérialisation et accès à la donnée

Maintenant que le **JsonDocument** est prêt, nous pouvons faire parcourir la longue chaîne de caractères contenant nos données avec la fonction **deserializeJson()**. Avec ArduinoJSON, il est conseillé de procéder de la façon suivante :

```
DeserializationError erreur = deserializeJson(doc, input);
```

Désérialiser un JSON

À ce stade, la variable `doc` contient la représentation mémoire du JSON. Il va être simple d'en extraire des valeurs comme par exemple :

```
// On utilise doc comme un dictionnaire
// Le "chartName" est une chaîne de caractère
const char* chartName = doc["chartName"];
// Le prix en EUR est un flottant
float price = doc["bpi"]["EUR"]["rate_float"];
```

Le code source nécessaire sera très similaire pour tout type de projet nécessitant d'accéder à un fichier JSON qu'il se trouve sur le web ou accessible via une API.

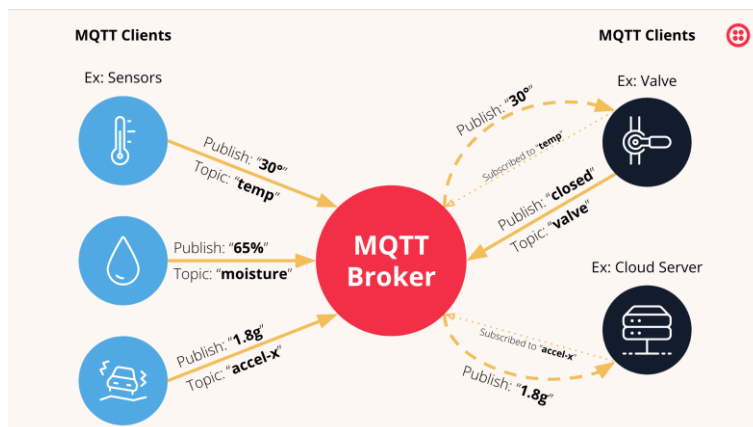


Envoyer de la donnée par MQTT

Introduction à MQTT

MQTT (Message Queuing Telemetry Transport) est un protocole de transport d'informations initialement développé par **IBM**. Il est pensé pour émuler un bus de données au dessus du protocole **TCP**, c'est-à-dire un moyen de diffuser de l'information à l'ensemble des équipements connectés sur le même bus, de la même manière qu'on le ferait sur un réseau industriel comme CANbus par exemple.

Introduction à MQTT



Introduction à MQTT

Les messages circulant sur un bus MQTT contiennent une charge utile (**payload**) pouvant être :

- du **texte** comme une valeur, une chaîne de caractères ou bien encore une structure de données comme un JSON,
- un **contenu binaire** comme une image ou un son, transmis en binaire dans la payload ou sous forme de chaîne de caractères encodée en base64 par exemple.

Introduction à MQTT

Chaque message est relié à un sujet (**topic**) permettant à un autre équipement de savoir rapidement s'il est intéressé ou non par ce message, sans avoir à le recevoir et l'analyser au préalable. C'est pour cette raison que MQTT est très utilisé dans l'IoT : il est conçu pour être très **léger** et permet de largement **soulager** l'objet connecté, souvent contraint en ressources CPU, mémoire, réseau et/ou énergie, en minimisant les messages qui lui sont adressés.

Introduction à MQTT



Exemple de topic MQTT

Un *topic* identifie la donnée dans une structure arborescence absolue où les niveaux sont séparés par des *slashes*. Le chemin étant toujours absolu, la racine est implicite; il ne faut donc pas placer le caractère « / » en début de *topic*.

Exemple : `ma_maison/chambre/temperature`

Dans cet exemple, il serait attendu de voir passer sur ce *topic* les données transmises à un capteur de température placé dans la chambre. Un système de chauffage disposé dans cette même chambre souscrirait à ce même *topic* pour décider si, oui ou non, il doit augmenter ou diminuer la température de cette pièce. Une interface pourrait également souscrire à ce *topic* pour afficher la température en cours dans cette pièce. Enfin, une base de données serait aussi à l'écoute pour enregistrer l'historique de la température dans cette pièce.

Introduction à MQTT

Utilisation dans l'environnement Arduino

Afin d'utiliser MQTT dans l'environnement Arduino, plusieurs bibliothèques sont disponibles. Ici, nous utiliserons **PubSubClient** ; en voici les principales méthodes.

Introduction à MQTT

Utilisation dans l'environnement Arduino

```

1 // Inclusion de la bibliothèque
2 #include <PubSubClient.h>
3
4 // Déclaration de l'objet mqtt_client et l'associer à la carte réseau
5 PubSubClient mqtt_client(wifiClient);
6
7 // Désignation du broker MQTT : nom d'hôte et port
8 mqtt_client.setServer("test.mqtt.com", 1883);
9
10 // Connexion au broker MQTT ; le 1er paramètre est l'id du client
11 mqtt_client.connect("abcdef123456", "login", "mot de passe");
12
13 // Donner du CPU à la machine d'état du client MQTT pour traiter les messages
14 mqtt_client.loop();
15
16 // Publier un message
17 mqtt_client.publish("ma_maison/chambre/temperature", "20");

```

Récupérer de la donnée par WebSocket

On peut souhaiter vouloir afficher directement les données d'un flux MQTT dans une page web mise à jour automatiquement dès qu'une nouvelle donnée est disponible. Le but est d'avoir une communication continue. Cela peut être utile pour un tableau de bord par exemple. En HTTP, le navigateur web envoie une requête au serveur qui envoie ensuite une réponse. Il nous faudrait alors effectuer une nouvelle requête pour chaque mise à jour souhaitée de la donnée. C'est un fonctionnement assez lourd, car il faudrait rafraîchir la page à intervalles réguliers. WebSocket est une solution. C'est un protocole de communication sur TCP qui permet une connexion persistante entre un navigateur web et un serveur. On peut donc avoir des échanges de données en temps réel.

Récupérer de la donnée par WebSocket

Protocole Web Socket

WebSocket est lié à HTTP dans la mesure où il l'utilise pour établir la connexion initiale avant de passer en WebSocket. Le protocole fonctionne de la façon suivante :

1. Établissement de la connexion (handshake) : le client envoie une requête spéciale au serveur pour établir une connexion WebSocket.
2. Handshake et réponse : si le serveur prend en charge les connexions WebSocket, la connexion est établie et restera ouverte jusqu'à ce qu'elle soit explicitement fermée soit par le client soit par le serveur.

Récupérer de la donnée par WebSocket

Protocole Web Socket

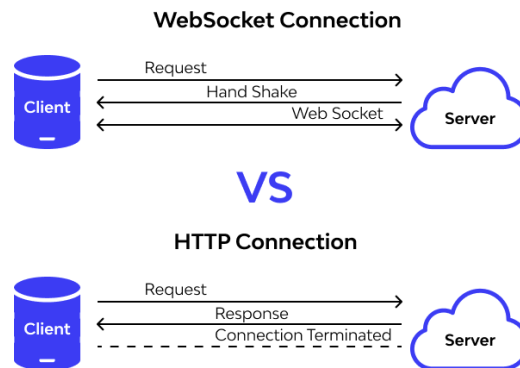
3. Communication bidirectionnelle : client et serveur peuvent s'envoyer des messages de manière bidirectionnelle en continu. Le client peut envoyer des messages au serveur et surtout vice versa sans avoir à attendre une demande du côté client.

4. Fermeture de la connexion : client et serveur peuvent décider de fermer la connexion WebSocket avec un message de fermeture spécial.

Nous observons au passage que les connexions WebSocket utilisent moins de bande passante et minimisent le trafic supplémentaire lors des échanges de messages.

Récupérer de la donnée par WebSocket

Protocole Web Socket



Récupérer de la donnée par WebSocket

MQTT sur WebSocket

On utilisera les WebSockets comme un tuyau dans lequel on va faire passer le flux MQTT venant du broker.

Il va nous falloir régler les deux extrémités du tuyau :

- le broker MQTT ;
- le navigateur web avec une librairie MQTT.

Récupérer de la donnée par WebSocket

MQTT sur WebSocket

Les dernières versions du broker Mosquitto permettent de se connecter en WebSocket. Cependant, la connexion n'est pas ouverte par défaut. Il faut alors l'activer dans le fichier *mosquitto.conf* en y ajoutant par exemple :

```
listener 1884
protocol websockets
allow_anonymous true
```

Cela crée un nouveau listener sur le port 1884 utilisant WebSocket.

Récupérer de la donnée par WebSocket

MQTT sur WebSocket

Du côté du navigateur web, vous pouvez utiliser le client JavaScript MQTT Paho de la fondation Eclipse 2 . Un code minimal serait :

```
<html>
<head>
  <title>IoT : MQTT sur WebSocket</title>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/paho-mqtt/1.0.1/
    mqttws31.js" type="text/javascript"></script>
  <script type="text/javascript" language="javascript">
    var mqtt;
    var host="localhost"; // URL du broker
    var port=1884; // port

    function onFailure(message) {
      console.log("Échec de connexion à"+host);
      setTimeout(MQTTconnect, 2000);
    }

    function onMessageArrived(msg){
      document.getElementById("data").innerHTML = msg.payloadString;
    }
  </script>
</head>
<body>
  <div id="data"></div>
</body>
</html>
```

Récupérer de la donnée par WebSocket

MQTT sur WebSocket

```
function onConnect() {
    mqtt.subscribe("app/#"); // topic
    document.getElementById("data").innerHTML ="Connecté à"+host;
}

function MQTTconnect() {
    // Comme la connexion est persistante, préciser un
    // identifiant client
    mqtt = new Paho.MQTT.Client(host, port, "nom_client");
    mqtt.onMessageArrived = onMessageArrived
    var options = {timeout:3, onSuccess:onConnect, onFailure:
        onFailure,};
    mqtt.connect(options);
}
</script>
</head>
<body>
<script>MQTTconnect();</script>
Message MQTT : <p id="data"></p>
</body>
```

Envoyer de la donnée en LoRaWAN

Principes des réseaux LoRaWAN

Le protocole LoRaWAN est un des protocoles sans-fil très utilisé dans l'IoT. Il permet des communications sécurisées, de longue portée (de l'ordre du kilomètre) et à faible consommation d'énergie, permettant l'alimentation d'un capteur sur de simples piles pendant des mois.

Envoyer de la donnée en LoRaWAN

Principes des réseaux LoRaWAN



La modulation LoRa

LoRaWAN utilise la modulation LoRa, qui permet une réception de messages jusqu'à -140dBm, c'est-à-dire une énergie dix mille fois plus petite que la plus petite des énergies recevables par un équipement Wi-Fi. De plus, les récepteurs LoRa sont capables d'aller chercher un message cohérent dans un bruit cent fois supérieur au message recherché. Ces propriétés sont exploitées pour permettre des transmissions de l'ordre de quelques kilomètres, avec pourtant une énergie nécessaire moindre que celle consommée par les émetteurs Wi-Fi. Il est à noter que cette robustesse est atteinte au prix d'un faible débit pour le transport des messages, ce qui n'est généralement pas gênant pour transporter de petites quantités de données comme des grandeurs issues de capteurs.

Envoyer de la donnée en LoRaWAN

Principes des réseaux LoRaWAN

Pour être mis en œuvre, un réseau LoRaWAN a besoin de plusieurs éléments serveurs : de réseau, d'application, etc. Une façon d'assurer la communication entre ces différents éléments consiste à utiliser, là encore, un bus MQTT.



Un réseau privé/libre ?

LoRaWAN communique dans les bandes ISM ; il est donc possible, pour tout un chacun, de déployer un réseau LoRaWAN privé sans demande d'autorisation, comme c'est le cas pour une borne Wi-Fi. Cependant, la technologie impose plusieurs éléments techniques, comme un serveur de réseau, qui peuvent être lourds à monter et à maintenir.

Envoyer de la donnée en LoRaWAN

Utilisation dans l'environnement Arduino

Là encore, afin d'utiliser les réseaux LoRaWAN dans l'environnement Arduino, l'écosystème donne accès à plusieurs bibliothèques. Ici, nous utiliserons MCCI LoRaWAN LMIC ; en voici les principales méthodes.

Envoyer de la donnée en LoRaWAN

Utilisation dans l'environnement Arduino

```
// Inclusion de LMIC
#include <lmic.h>

// Identifiants et clés pour se connecter au réseau LoRaWAN
static const u1_t PROGMEM APPEUI[8]={ 0,0,0,0,0,0,0,0 };
static const u1_t PROGMEM DEVEUI[8]={ 0,0,0,0,0,0,0,0 };
static const u1_t PROGMEM APPKEY[16]={0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};

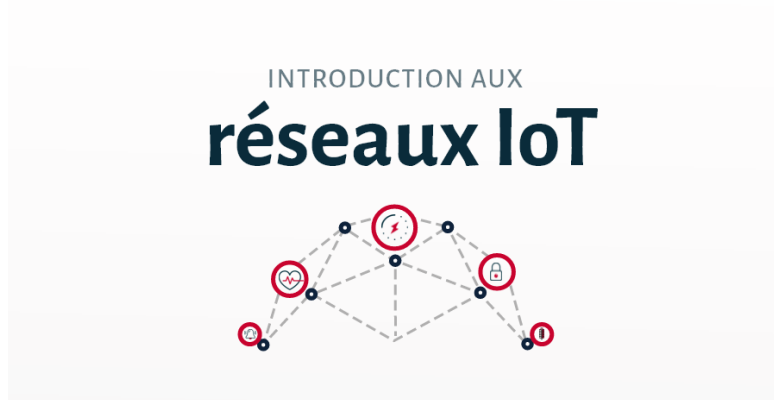
// Connexion au réseau LoRaWAN
os_init(); LMIC_reset();

// Donner du CPU a la machine d'état LoRaWAN pour traiter les messages
os_runloop_once();

// Envoyer un message
LMIC_setTxData2(1, mydata, sizeof(mydata)-1, 0);
```

Introduction aux réseaux IoT

une vue d'ensemble



Introduction aux réseaux IoT

une vue d'ensemble

Pour connecter un objet à Internet, plusieurs choix s'offrent à vous. Vous en connaissiez déjà certains : le **Wifi**, le **Bluetooth**, le réseau cellulaire (3G, 4G). Ce sont des modes de connexion extrêmement répandus et grand public.



Introduction aux réseaux IoT

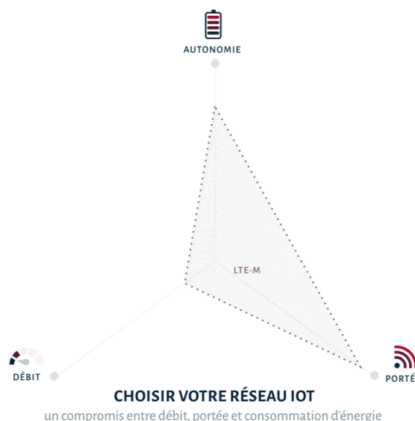
Les 3 éléments fondamentaux d'un réseau IoT

Chaque réseau dispose de ses points forts et de ses points faibles. Cependant, chaque technologie peut être considérée en fonction de trois critères :

- Sa **consommation d'énergie** (combien d'énergie consommée pour envoyer 1 Mo).
- Sa **bande passante** (combien de temps nécessaire pour envoyer 1 Mo).
- Sa **portée** (sur combien de mètres ou kilomètres la connexion reste fiable).

Introduction aux réseaux IoT

Les 3 éléments fondamentaux d'un réseau IoT



Introduction aux réseaux IoT

Les 3 éléments fondamentaux d'un réseau IoT

Ex.

renons l'exemple d'une caméra embarquée dans une voiture, appelée dashcam. Nous avons besoin de récupérer le film enregistré uniquement en cas d'accident.

Les dashcams connectées n'envoient donc le film à votre assureur qu'en cas de choc détecté, et suppriment les vidéos enregistrées au fur et à mesure le reste du temps.



Quel réseau IoT choisir pour connecter vos objets ?

Passons en revue l'ensemble des technologies permettant à votre objet d'être connecté à Internet, en les classant en deux catégories :

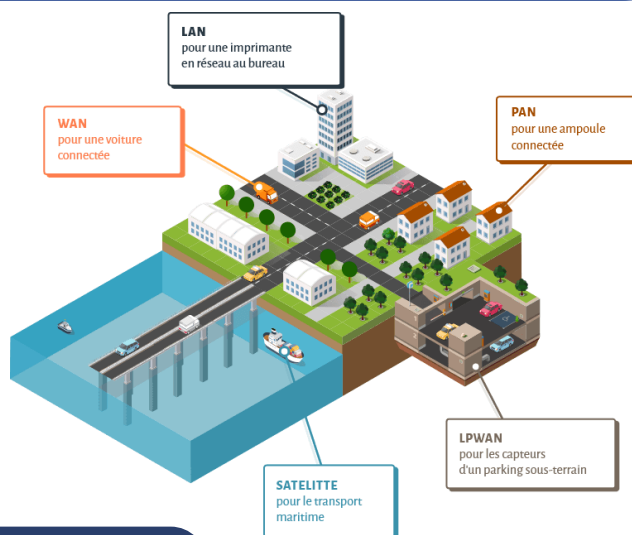
1. Réseaux « **courte distance** » : de quelques centimètres à quelques mètres.
2. Réseaux « **longue distance** » : de quelques dizaines de mètres à plusieurs kilomètres.

Quel réseau IoT choisir pour connecter vos objets ?

Nous pouvons différencier les types de réseaux IoT selon cette typologie :

- **WAN (Wide Area Network)** : un réseau de plusieurs dizaines de kilomètres carrés
- **LPWAN (Low Power Wide Area Network)** : réseau de plusieurs dizaines de kilomètres carrés mais utilisant peu d'énergie (car peu de bande passante)
- **PAN (Personal Area Network)** : le réseau de quelques mètres (Bluetooth)
- **LAN (Local Area Network)** : le réseau Internet privé de votre domicile ou de votre entreprise (Wifi)
- **Satellite** : partout dans le monde pour peu de ne pas être dans un tunnel (GPS)

Quel réseau IoT choisir pour connecter vos objets ?



1. Les réseaux IoT courte distance



Bluetooth

Probablement la technologie la plus grand public, démocratisée depuis le début des années 2000.

Sa consommation d'énergie est relativement faible ainsi que sa portée (environ 10 mètres).

Le **Bluetooth** possède une bande passante intermédiaire (entre 1 et 3 Mb/s) mais amplement suffisante pour la plupart des utilisations classiques.

1. Les réseaux IoT courte distance



Bluetooth

A titre d'exemple, vous pouvez streamer de la musique en MP3 mais difficile de relayer un film en HD. Le Bluetooth est un choix approprié pour beaucoup de MedTech (médecine).

Par exemple, la société DiabNext permet de surveiller son diabète à l'aide de capteurs connectés au téléphone en Bluetooth.



1. Les réseaux IoT courte distance



Zigbee

Le **Zigbee** est un protocole de communication similaire au Bluetooth mais dédié à l'IoT. Il consomme peu, et est fait pour envoyer de petits volumes de données (**entre 20 et 250 Kb/s**). Il permet d'utiliser chaque objet comme "rallonge de connexion".

1. Les réseaux IoT courte distance



Zigbee

Par exemple, l'ampoule connectée de votre salon va permettre de connecter l'ampoule de votre chambre, comme un relai jusqu'au "hub", branché au réseau Internet. Les objets connectés de Ikea ou Philips Hue utilisent Zigbee.

Il s'agit d'envoyer en effet des ordres, tels que « allumer la lumière de la chambre », peu générateurs de données lourdes.



1. Les réseaux IoT courte distance



Wifi

Le Wifi permet un très important débit data (environ **400 Mb/s** pour les modems grand public récents), de manière fiable et sécurisée. Toutefois il consomme beaucoup d'énergie et sa portée est relativement limitée (**35 m**), d'autant plus au travers d'obstacles comme des murs épais.

1. Les réseaux IoT courte distance

Wifi 6

Le Wifi 6, aussi appelé Wifi AX, nouvelle génération en cours de commercialisation, va permettre d'améliorer considérablement la connectivité des objets connectés.

Avec un débit de 10 Gb/s, il permettra de gérer beaucoup plus d'appareils plus efficacement tout en consommant moins d'énergie : il s'agit de limiter le mode veille des objets en les "réveillant" lorsqu'un échange de données est nécessaire.

1. Les réseaux IoT courte distance

Wifi 6



1. Les réseaux IoT courte distance

RFID

Un mode de connexion particulier, dédié à l'identification, on pourrait le surnommer code-barre 2.0.

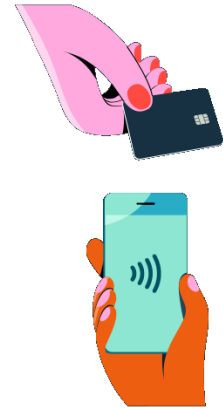
Une puce RFID permet d'identifier à distance, comme on le ferait en scannant un code-barre, mais jusqu'à 100 mètres. C'est une technologie qui vient en complément d'autres types de connectivité.



1. Les réseaux IoT courte distance

NFC (Near Field Communication)

Son utilisation la plus connue est le paiement sans contact. Le NFC peut échanger un très petit volume de données sur une distance très rapprochée (quelques centimètres). C'est une technologie largement répandue pour les ouvertures de portes dans les chambres d'hôtels. Elle permet aussi d'augmenter l'expérience utilisateur de manière ludique.



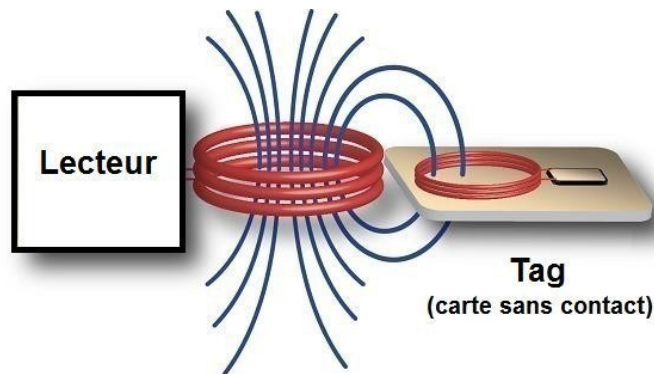
1. Les réseaux IoT courte distance

NFC (Near Field Communication)



1. Les réseaux IoT courte distance

NFC (Near Field Communication)



2. Les réseaux IoT longue distance

Réseaux cellulaires

La technologie qui équipe nos téléphones portables depuis 30 ans. Plusieurs générations se sont succédé :

- GSM : supportant uniquement les appels et SMS
- 2G : rendant possible l'envoi de MMS
- 3G : initiant l'utilisation de l'Internet mobile
- 4G : permettant le haut débit sur mobile, par exemple le streaming vidéo HD)
- 5G en cours de commercialisation (particulièrement adapté à l'IoT très gourmand en data, comme les voitures autonomes).

2. Les réseaux IoT longue distance

Réseaux cellulaires

1G
1970

2G
1980

3G
2000

4G
2012

5G
2020

2. Les réseaux IoT longue distance

Réseaux cellulaires

Pour connecter votre objet au réseau cellulaire, il faut l'équiper, comme un téléphone, d'une carte SIM. Cependant, les cartes SIM M2M ne sont pas celles vendues pour le grand public. Elles sont appelées cartes M2M, machine-to-machine.

Elles ont plusieurs spécificités, par exemple de pouvoir résister à des températures extrêmes.



2. Les réseaux IoT longue distance

Réseaux cellulaires

L'utilisation du réseau cellulaire pour l'IoT présente de nombreux avantages :

- Les antennes sont déjà installées et dense.
- La couverture d'une antenne s'étend sur plusieurs dizaines de kilomètres.
- La configuration est minimale.
- La part de la connectivité dans le coût de l'IoT est beaucoup plus faible que pour d'autres technologies.

2. Les réseaux IoT longue distance

Réseaux cellulaires

L'utilisation du réseau cellulaire pour l'IoT présente de nombreux avantages :

- Les antennes sont déjà installées et dense.
- La couverture d'une antenne s'étend sur plusieurs dizaines de kilomètres.
- La configuration est minimale.
- La part de la connectivité dans le coût de l'IoT est beaucoup plus faible que pour d'autres technologies.

2. Les réseaux IoT longue distance

Réseaux cellulaires

Enfin, le cellulaire est aussi un mode de **connectivité beaucoup plus sécurisé**. C'est donc une solution particulièrement adaptée aux services publics, comme la connexion de bornes de **vélos en libre-service** ou encore dans le domaine **de la santé** et de la **sécurité**.

2. Les réseaux IoT longue distance

LPWAN non cellulaire

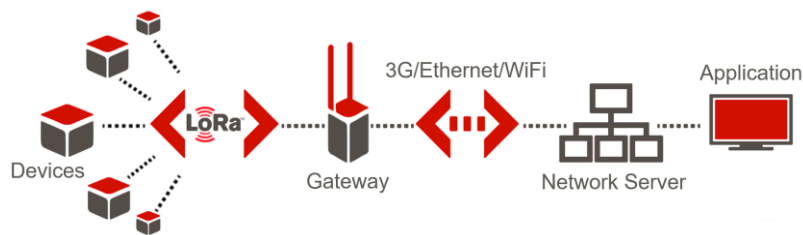
Il s'agit d'envoyer de très petits volumes de données, entre **300 bits/s** et **50 kbit/s**, sur une distance pouvant atteindre jusqu'à **40 kilomètres** et **à travers des obstacles (murs, sous terre)** en consommant le moins d'énergie possible.



2. Les réseaux IoT longue distance

LPWAN non cellulaire

La technologie IoT **longue distance** repose sur le fait que les normes et standards utilisés sont partagés par le plus d'acteurs possible, il existe plusieurs systèmes concurrents, **sous** ou **sans licence**.



2. Les réseaux IoT longue distance

LPWAN non cellulaire

Les technologies sans licence sont présentes sur un spectre de fréquences "libres", à la manière des radios amateurs ou des talkies-walkies. Celles sous licence nécessitent une carte SIM, car les fréquences sont attribuées par les pouvoirs publics aux opérateurs (Orange, IAM, INWI...).

Attention

A noter que ce n'est pas parce qu'une technologie est sans licence qu'elle peut être utilisée de manière libre. Pour que votre objet soit connecté aux réseaux **Sigfox** ou **LoRa**, il est nécessaire d'avoir une puce compatible.

2. Les réseaux IoT longue distance

LPWAN non cellulaire

Sigfox (société française) est un réseau de communication ultra bas débit dans plus de 70 pays. Les objets connectés compatibles peuvent communiquer de très faibles volumes de data (12 octets) au maximum 140 fois par jour.



2. Les réseaux IoT longue distance

LPWAN non cellulaire

Le concurrent principal de Sigfox, l'alliance LoRa, est une association de plusieurs acteurs de l'IoT dans le monde, dont les objets sont connectés via la technologie LoRaWan, basée sur (et uniquement sur) une puce électronique Semtech.



2. Les réseaux IoT longue distance

LPWAN non cellulaire

Weightless est un standard open source. Son adoption est plus confidentielle que ses deux précédents concurrents, car son déploiement plus récent. Il fait plus de sens pour des projets relativement sophistiqués ainsi que dans les cas où les débits ascendants et descendants sont tous deux importants.



2. Les réseaux IoT longue distance

LPWAN non cellulaire

Ces trois systèmes, parmi de nombreux autres LPWAN, sont principalement dédiés à l'Internet des objets de type capteurs et senseurs. Il s'agit d'envoyer de très faibles volumes de data (généralement entre 300 bits/s et 50 kbit/s) :

- Température dépassant un certain niveau
- Changement anormal de pression dans une canalisation
- Mouvement imperceptible sur une structure telle qu'un pont
- Place de parking souterrain libre/occupée

2. Les réseaux IoT longue distance

LPWAN non cellulaire

La technologie LPWAN est donc parfaitement adaptée à ce type de mission, mais très limitée par la bande passante. S'il s'agit de communiquer des données plus lourdes (telles que des photos ou vidéos), le réseau cellulaire s'impose.

2. Les réseaux IoT longue distance

LPWAN cellulaire

Contrairement aux systèmes sans licence évoqués précédemment, les technologies LTE-M et NB-IoT consistent en une évolution de l'utilisation des antennes 4G existantes, et ne nécessitent pas la pose de nouvelles antennes.

2. Les réseaux IoT longue distance

LPWAN cellulaire

Dans tous les cas, ces technologies permettent d'échanger sur le réseau cellulaire, donc avec une couverture réseau extrêmement solide et à travers plusieurs pays.

La batterie d'un capteur connecté grâce à ces technologies peut tenir entre 5 et 10 ans.

2. Les réseaux IoT longue distance

LPWAN cellulaire

La technologie LTE-M permet l'échange de faibles quantités de données pour une consommation énergétique très restreinte. Toutefois, c'est le dispositif LPWAN qui permet la plus grosse bande passante, en plus de la voix et le SMS.

2. Les réseaux IoT longue distance

LPWAN cellulaire

Le fait que la masse de données échangées est significativement supérieure aux autres dispositifs LPWAN, et qu'il intègre le roaming, en fait une solution de choix pour tous les dispositifs médicaux de surveillance et autres IoT wearables (montres et bracelets connectés).

2. Les réseaux IoT longue distance

LPWAN cellulaire

Dispositif cousin du LTE-M, mais un débit data 4 fois plus faible (0,24 kb/s contre 1 Mo/s), le réseau Nb-IoT correspond à un usage de type : capteurs de fuite d'eau, capteurs d'acidité pour les sols agricoles, capteurs de pollution de l'air.

2. Les réseaux IoT longue distance

LPWAN cellulaire

C'est-à-dire l'envoi de données très légères (quelques chiffres) à intervalles réguliers, et sur un capteur statique, qui peut-être enterré sous plusieurs mètres (canalisations, parking souterrain, silo agricole, cave viticole...).

2. Les réseaux IoT longue distance

Satellite

Le réseau satellite reste encore l'unique solution pour les zones reculées (haute mer, désert, haute montagne), où il n'y a aucune alternative.

Les entreprises communément appelées du New Space (Space X, Blue Origin...) développent des projets de constellations qui ont pour but de connecter toute la planète à un réseau Internet haut débit.



